

Simulador *Cycle-Accurate* de uma Rede-em-Chip SPIN Topologia Fat-Tree Quaternária em SystemC

Comutação Wormhole, Controle de Fluxo por Créditos e Arbitragem Probabilística

Breno Barbosa de Oliveira Felipe Marley de Oliveira Gomes Fernando Gomes de
Almeida Gustavo de Freitas Rodrigues

Curso de Ciência da Computação
Departamento de Informática e Matemática Aplicada (DIMA)
Universidade Federal do Rio Grande do Norte

Trabalho Prático 2 — Redes-em-Chip (NoC)

- 1 Introdução
- 2 Implementação e Diagrama de Blocos
- 3 Resultado de Simulação
- 4 Conclusões

Introdução — Por que Redes-em-Chip?

- SoCs modernos integram dezenas a centenas de núcleos em um mesmo chip.
- **Barramentos** não escalam (banda, energia); **fios dedicados** explodem em área de roteamento.
- **Rede-em-Chip (NoC)**: comunicação comutada por pacotes, com roteadores e enlaces — como uma rede de computadores em miniatura.

Objetivo do trabalho

Projetar, implementar e **verificar por execução** um simulador *cycle-accurate* (nível RTL) da NoC **SPIN**, em **SystemC**, com topologia *fat-tree* quaternária.

Três mecanismos de hardware:

- **Wormhole:** mensagem dividida em *flits* (HEADER/PAYLOAD/TAIL).
- **Créditos:** só transmite se há espaço no *buffer* vizinho $\Rightarrow$ sem perda de flit.
- **Árbitro probabilístico:** sorteio a cada ciclo evita *starvation*.

Roteamento em duas fases:

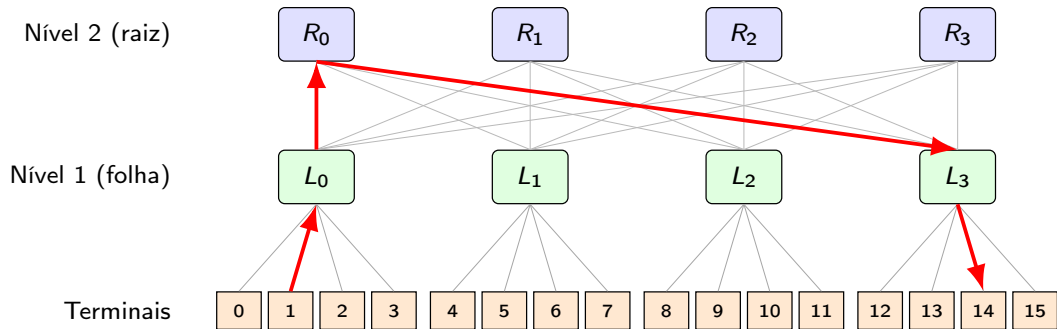
- **Subida adaptativa:** escolhe o enlace ascendente com mais créditos livres.
- **Descida determinística:** caminho único dado pelo endereço hierárquico do destino.

- **SystemC 3.0.2** (Accellera), C++17, g++ 16.
- Modelo *cycle-accurate*: processos SC_METHOD sensíveis à borda do relógio (10 ns, 100 MHz).
- **8 roteadores** (4 folha + 4 raiz) e **16 terminais**.
- *Buffers* FIFO de profundidade 4 por porta.
- Máquina *wormhole* com tabela de travas (input_locks).

Módulos

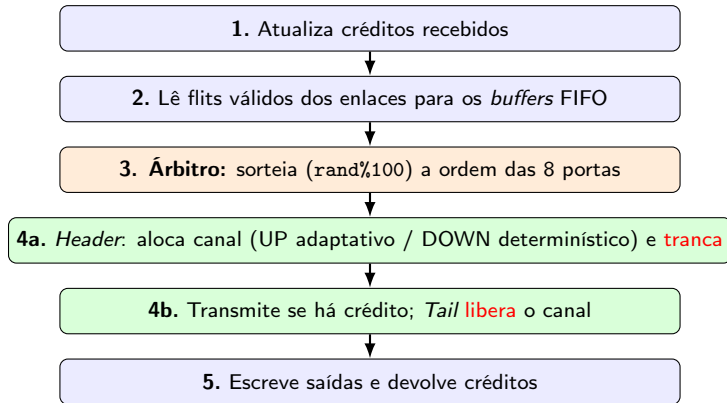
spin_flit	tipo do flit
terminal	injeta/recebe
rspin_router	roteador + árbitro
spin_fat_tree	topologia (top)
main	testbench

Diagrama de Blocos — Topologia Fat-Tree



Em **vermelho**: caminho do exemplo $T_1 \rightarrow L_0 \rightarrow R_0 \rightarrow L_3 \rightarrow T_{14}$.

Diagrama de Blocos — Pipeline do Roteador



Executado por `routing_logic` a cada borda de relógio.

Resultado — Cenário e Log de Execução Real

Cenário: `configure_traffic(1,14)` — T_1 envia 4 flits a T_{14} (sub-árvores distintas). Saída de console **copiada da execução**:

```
[ PACOTE INICIADO ] @ 30 ns | Origem: 1 Destino: 14
[ARBITRO] Roteador Nivel 1 ID 0 | caiu em 50% (Prioridade UP) →Atendendo Entrada 1
→[HW] Roteamento Adaptativo trancou porta UP 0
>@ 40 ns | Roteador Nivel 1 (ID 0) despachou [HEADER] →Porta UP 0
>@ 50 ns | Roteador Nivel 2 (ID 0) despachou [HEADER] →Porta DOWN 3
>@ 60 ns | Roteador Nivel 1 (ID 3) despachou [HEADER] →Porta DOWN 2
<< @ 70 ns | Terminal 14 RECEBEU [HEADER] | Dado: >INICIO_DA_MENSAGEM <
<< @ 80 ns | Terminal 14 RECEBEU [PAYLOAD] | Dado: Ola mundo :) #1
<< @ 90 ns | Terminal 14 RECEBEU [PAYLOAD] | Dado: Ola mundo :) #2
<< @ 100 ns | Terminal 14 RECEBEU [TAIL] | Dado: >FIM_DA_MENSAGEM <
→(Pacote completamente recebido e remontado!)
```

Resultado — Pipeline Sistólico Wormhole

Ocupação de cada enlace por ciclo (reconstruída do log real):

t (ns)	$T_1 \rightarrow L_0$	$L_0 \rightarrow R_0$	$R_0 \rightarrow L_3$	$L_3 \rightarrow T_{14}$
40	PAY#1	HEADER	—	—
50	PAY#2	PAY#1	HEADER	—
60	TAIL	PAY#2	PAY#1	HEADER
70	—	TAIL	PAY#2	PAY#1
80	—	—	TAIL	PAY#2
90	—	—	—	TAIL

A diagonal evidencia o avanço *wormhole*: 1 salto por ciclo. Latência do header = 4 ciclos; pacote completo entregue em 100 ns.

Resultado — Bateria de Casos de Teste (CTest)

Testes autoverificáveis: cada cenário compara os pacotes remontados com o esperado.

Cenário	Fluxos	Verifica	Status
Fluxo único	$T_1 \rightarrow T_{14}$	caminho completo via raiz	OK
Fluxo local	$T_0 \rightarrow T_3$	roteamento na mesma folha	OK
Convergência	$T_0, T_8 \rightarrow T_7$	sem perda sob contenção	OK
Permutação	8 fluxos disjuntos	N fluxos sem <i>deadlock</i>	OK
Muitos-p/-um	$T_0..T_3 \rightarrow T_{10}$	serialização justa	OK

Resultado: 5/5 testes aprovados. A convergência confirma a correção do defeito de contenção (perda de cauda quando dois fluxos disputam a saída).

Conclusões

- Implementamos e **executamos** um simulador *cycle-accurate* da NoC SPIN (*fat-tree* quaternária) em SystemC.
- Validamos os três mecanismos centrais:
 - **Wormhole**: alocação no *header*, fluxo do corpo, liberação no *tail*;
 - **Créditos**: entrega sem perda de flits;
 - **Roteamento**: subida adaptativa + descida determinística;
 - **Árbitro**: intervenção probabilística a cada salto.
- A verificação por execução **expôs e corrigiu um bug real** (operador `==` ignorava o campo `data`, fazendo o `sc_signal` suprimir o 2º *payload*).

Trabalhos futuros

Múltiplos fluxos concorrentes; métricas de latência/vazão; geração de *traces* VCD; cargas sintéticas (uniforme, *hotspot*).

Obrigado!

Perguntas?

Breno Felipe Fernando Gustavo — UFRN/DIMAp